

SESSION 5

EXPERIMENT – 9

Named Pipes (FIFO) for Inter-Process Communication

Aim

To implement **Inter-Process Communication using Named Pipes (FIFO)** and demonstrate communication between processes using `mkfifo()`, file descriptors, and non-blocking I/O.

Theory

1. Named Pipes

A **Named Pipe**, also called a **FIFO (First-In-First-Out)**, is an IPC mechanism that allows unrelated processes to communicate with each other.

Unlike an anonymous pipe, a named pipe has a **name in the filesystem** and can be accessed by different processes.

For example:

```
/tmp/app_log.fifo
```

The FIFO can be created using:

```
mkfifo("/tmp/app_log.fifo", 0666);
```

2. Anonymous Pipe vs Named Pipe

Anonymous Pipe	Named Pipe
Usually used between related processes	Can be used by unrelated processes
Does not have a filesystem name	Has a filesystem name
Created using <code>pipe()</code>	Created using <code>mkfifo()</code>
Temporary communication mechanism	FIFO exists in the filesystem until removed

3. `mkfifo()`

The `mkfifo()` system call creates a named pipe in the filesystem.

Syntax:

```
mkfifo("filename", permissions);
```

Example:

```
mkfifo("/tmp/app_log.fifo", 0666);
```

Here:

- `/tmp/app_log.fifo` → name of the FIFO
- `0666` → read/write permissions

4. FIFO Communication

Communication takes place through the FIFO file.

Application 1

```
|  
| write()  
↓  
/tmp/app_log.fifo  
|  
| read()
```



Logging Daemon

Multiple applications can write their log messages into the FIFO, while a logging daemon continuously reads and processes them.

5. Non-Blocking I/O

Normally, reading from a FIFO can block until data becomes available.

Using:

`O_NONBLOCK`

allows a process to perform **non-blocking I/O**.

For example:

```
open("/tmp/app_log.fifo", O_RDONLY | O_NONBLOCK);
```

If no data is currently available, the process does not remain permanently blocked.

6. File Permissions

FIFO permissions determine which users and processes can access the pipe.

For example:

`0666`

provides read and write permissions according to the system's permission rules.

In real applications, permissions should be configured carefully so unauthorized users cannot inject or read sensitive information.

7. Security Considerations

Named pipes can create security risks if they are placed in locations accessible to untrusted users.

Security measures include:

- Restricting FIFO permissions.
- Placing the FIFO in a controlled directory.
- Validating incoming data.
- Removing the FIFO when it is no longer required.
- Preventing unauthorized processes from writing sensitive or malicious data.

Algorithm

1. Define the path of the named pipe.
2. Create the FIFO using `mkfifo()`.
3. Open the FIFO for reading.
4. Use non-blocking mode with `O_NONBLOCK`.
5. Continuously check for incoming messages.
6. Read and display the log messages.
7. Close the FIFO.
8. Remove the FIFO using `unlink()`.

Code

```
C exp9.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <sys/stat.h>
6  #include <string.h>
7  #include <errno.h>
8
9  #define FIFO_PATH "/tmp/app_log.fifo"
10
11 int main() {
12
13     char buffer[256];
14
15     // Create named pipe
16     if (mkfifo(FIFO_PATH, 0666) == -1) {
17         if (errno != EEXIST) {
18             perror("mkfifo");
19             return 1;
20         }
21     }
22
23     printf("Logging daemon started.\n");
24     printf("Waiting for log messages...\n");
25
26     // Open FIFO in non-blocking read mode
27     int fd = open(FIFO_PATH, O_RDONLY | O_NONBLOCK);
28
29     if (fd == -1) {
30         perror("open");
31         unlink(FIFO_PATH);
32         return 1;
33     }
```

```

32         return 1;
33     }
34
35     while (1) {
36
37         ssize_t bytes = read(fd, buffer, sizeof(buffer) - 1);
38
39         if (bytes > 0) {
40
41             buffer[bytes] = '\0';
42
43             printf("Log received: %s", buffer);
44             fflush(stdout);
45
46         } else if (bytes == -1 && errno != EAGAIN &&
47                  errno != EWOULDBLOCK) {
48
49             perror("read");
50             break;
51         }
52
53         sleep(1);
54     }
55
56     close(fd);
57     unlink(FIFO_PATH);
58
59     return 0;
60 }

```

OUTPUT:

```

chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ echo "ERROR: Database connection failed" > /tmp/app_log.fifo
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ echo "ERROR: File not found" > /tmp/app_log.fifo
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ 

```

```

chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ gcc exp9.c -o exp9
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ./exp9
Logging daemon started.
Waiting for log messages...
Log received: ERROR: Database connection failed
Log received: ERROR: File not found

```

EXPERIMENT – 10

Linux Signals, Process Sessions and Daemon Processes

Aim

To study and implement **POSIX signals**, signal generation and handling, process sessions using `setsid()`, and the basic structure of a daemon process.

Theory

1. Signals

A **signal** is a software notification sent to a process to inform it that a particular event has occurred.

Signals are used by the operating system and processes for communication and process control.

Examples include:

Signal	Purpose
SIGINT	Interrupt process
SIGTERM	Request process termination
SIGKILL	Forcefully terminate process
SIGSTOP	Stop/suspend process
SIGCONT	Continue a stopped process
SIGSEGV	Segmentation fault
SIGCHLD	Child process has changed state

2. SIGINT

SIGINT is normally generated when the user presses:

Ctrl + C

It requests the process to terminate.

A program can catch SIGINT and execute a custom signal handler.

3. SIGTERM

SIGTERM is a request for a process to terminate gracefully.

It can be sent from the terminal using:

```
kill -SIGTERM PID
```

Unlike SIGKILL, a program can catch and handle SIGTERM.

4. SIGKILL

SIGKILL immediately terminates a process.

It cannot be caught, blocked, or ignored by the process.

For this reason, programs should generally be given an opportunity to terminate gracefully using SIGTERM first.

5. Signal Handler

A **signal handler** is a function that is executed when a process receives a particular signal.

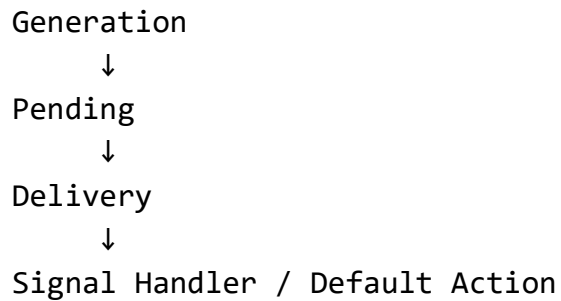
For example:

```
void handle_signal(int sig)
{
    printf("Signal received\n");
}
```

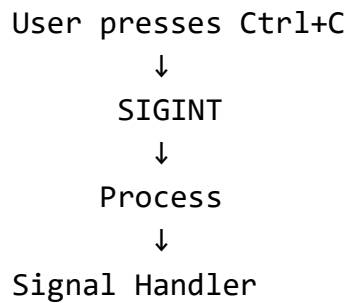
A signal can be associated with a handler using functions such as `signal()`.

6. Signal Lifecycle

A signal generally passes through these stages:



For example:



7. Process Sessions

A **session** is a collection of process groups.

A process can create a new session using:

`setsid();`

The process calling `setsid()` becomes the **session leader** of the newly created session.

This is especially important when creating daemon processes.

8. Daemon Process

A **daemon** is a background process that normally runs without being directly attached to a terminal.

Examples of Linux services include:

- Web servers
- Logging services
- Scheduling services
- Background system services

A typical daemon performs operations such as:

1. Forking from the parent.
2. Creating a new session using `setsid()`.
3. Detaching from the controlling terminal.
4. Continuing execution in the background.

Algorithm

1. Register handlers for `SIGINT` and `SIGTERM`.
2. Display the process ID.
3. Keep the process running and wait for signals.
4. Send a signal using another terminal.
5. Execute the corresponding signal handler.
6. When `SIGTERM` is received, terminate gracefully.
7. Demonstrate session creation using `setsid()`.

Code:

```
C exp10.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <signal.h>
5
6  void handle_signal(int sig) {
7
8      if (sig == SIGINT) {
9          printf("\nSIGINT received.\n");
10         printf("Use SIGTERM to terminate the program gracefully.\n");
11     }
12
13     else if (sig == SIGTERM) {
14         printf("\nSIGTERM received.\n");
15         printf("Process terminating gracefully...\n");
16         exit(0);
17     }
18 }
19
20 int main() {
21
22     printf("Process started.\n");
23     printf("PID: %d\n", getpid());
24     printf("Parent PID: %d\n", getppid());
25     printf("Process Group ID: %d\n", getpgrp());
26
27     // Register signal handlers
28     signal(SIGINT, handle_signal);
29     signal(SIGTERM, handle_signal);
30
31     printf("\nWaiting for signals...\n");
32
33     while (1) {
34         pause();
35     }
36
37     return 0;
38 }
```

OUTPUT:

```
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ./exp10
Process started.
PID: 7419
Parent PID: 7326
Process Group ID: 7419

Waiting for signals...

SIGTERM received.
Process terminating gracefully...
```

```
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ./exp10
Process started.
PID: 7360
Parent PID: 7243
Process Group ID: 7360

Waiting for signals...
^C
SIGINT received.
Use SIGTERM to terminate the program gracefully.
█
```

Result:

Thus, POSIX signals were successfully implemented and handled using SIGINT and SIGTERM. The program successfully responded to signals and terminated gracefully on receiving SIGTERM. Process information such as PID, PPID, and Process Group ID was observed. The `setsid()` system call was also successfully used to demonstrate the creation of a new process session.